PES UNIVERSITY

100 feet Ring Road, BSK 3rd Stage, Bengaluru 560085

Department of Computer Science and Engineering

(AI & ML)

B. Tech. CSE – 5th Semester

Aug – Dec 2026

UE23CS351A

DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT on

FEASTLINK – Surplus Food Redistribution System for Functions and Shelter Homes

Submitted by:

SRN	Name	Section
PES1UG24AM254	Saransh Mittal	5E
PES1UG24AM300	T Surya	5E

Class of Prof. Junaid Rasool

Lab Faculty: Junaid Rasool

Abstract

FEASTLINK is a web-based surplus-food redistribution platform designed to connect event organizers who have excess prepared food with shelter homes that require meals. The system supports a structured workflow covering organizer registration, event creation, surplus-food listing, shelter browsing, food requests, approval, allocation, delivery tracking, and donation-impact recording. The project uses MySQL 8.0 as its primary relational database for structured transactional records and MongoDB 8.0 for flexible activity logs and notifications. The application is implemented using Next.js 14, React, TypeScript, and database drivers for MySQL and MongoDB. Its relational design contains eight principal tables connected through primary-key and foreign-key constraints. Additional DBMS features documented in the project include joins, a view, indexes, a trigger, a stored procedure, a function, and transaction handling. FEASTLINK aims to reduce avoidable food wastage while maintaining traceable records of food distribution and beneficiaries. The report follows the prescribed DBMS project template and documents the design, implementation, and observed test workflow.

Table of Contents

Sl. No.	Topic	Page No.
1	Introduction – Problem Statement	4
2	User Requirements Specification – Tech Stack	5
3	E-R Model: entities, attributes and ER diagram	6
4	Relational Model and Schema Diagram	7
5	SQL DDL Statements	8
6	SQL DML Statements	8
7	NoSQL Database Commands	8
8	Results – Resulting Tables / Testing	9
9	Next.js Project Tree Structure	9
10	Application Code and Demo Screenshots	9
11	Evaluation Rubrics – Completion Status	9

Introduction

1. Problem Statement

Large functions, celebrations, and organized events can generate surplus prepared food, while shelter homes may face an ongoing need for meals. In the absence of a coordinated mechanism, usable food can be discarded and potential beneficiaries may remain unserved. FEASTLINK addresses this gap through a centralized web application where organizers publish surplus-food details and shelter administrators request suitable quantities. The system manages approval, allocation, delivery status, and donation outcomes, providing a structured record of the redistribution process.

Project Objectives

Reduce avoidable surplus-food wastage by making available food visible to eligible shelter homes.

Connect event organizers and shelter administrators through a unified platform.

Maintain structured and referentially consistent records of users, events, food listings, requests, allocations, deliveries, and donation history.

Track requests, approval decisions, allocation quantities, and delivery progress.

Record meals served and beneficiary counts to support impact reporting.

Demonstrate relational DBMS concepts together with document-based activity logs and notifications.

Scope

The project scope covers organizer and shelter workflows, core database operations, and impact records. The documented application uses two database systems with distinct responsibilities: MySQL for structured and transactional data, and MongoDB for flexible log and notification documents. The report describes features present in the project handoff and DBMS mini-project documentation.

User Requirements Specification

Functional Requirements

Users can register and log in under supported roles (ORGANIZER, SHELTER_ADMIN, and ADMIN).

Organizers can create events and publish surplus-food listings with type, quantity, meal estimate, preparation time, availability deadline, and status.

Shelter administrators can maintain shelter profiles, browse available food, submit requests, and view request/delivery status.

Organizers can approve or reject incoming requests; approved requests are represented by food-allocation records.

Delivery records track pickup, delivery time, delivery status, and notes.

Donation history records meals served, completion date, beneficiary count, and remarks.

The application generates activity logs and notifications and displays donation-impact statistics.

Technology Stack and Justification

Technology	Use and justification
Next.js 14 / React	Frontend pages and backend API routes in one application framework.
TypeScript	Typed implementation language for application logic and API integration.
MySQL 8.0	Relational storage for structured entities, foreign-key relationships, constraints, joins, and transactional records.
MongoDB 8.0	Document storage for flexible activity logs and notifications whose fields may vary.
mysql2	Node.js driver used to connect the application to MySQL.
MongoDB Node.js Driver	Application connectivity to MongoDB collections.
MySQL Workbench / MongoDB Compass	Database administration and inspection tools.

NoSQL choice: MongoDB is used for activity logs and notifications in the feastlink_logs database. These records are naturally document-oriented and can include flexible details, action metadata, user identifiers, timestamps, and read state. MySQL remains the source for core relational and transactional application data.

Architecture

User → Next.js frontend → Next.js API routes → MySQL and/or MongoDB → response to frontend.

E-R Model

List of Entities

Entity	Description / key attributes	Weak entity?
USERS	user_id (PK), name, email, phone, password, role, created_at	No
EVENTS	event_id (PK), organizer_id (FK), event_name, event_type, event_date, location, expected_guests	No
SHELTER_HOMES	shelter_id (PK), user_id (FK), shelter_name, location, contact_person, phone, number_of_residents, food_preference, availability_status	No
SURPLUS_FOOD	food_id (PK), event_id (FK), food_name, food_type, quantity, estimated_meals, prepared_time, available_until, status	No
FOOD_REQUESTS	request_id (PK), food_id (FK), shelter_id (FK), requested_meals, request_date, status	No
FOOD_ALLOCATIONS	allocation_id (PK), request_id (FK), allocated_meals, allocation_date, status	No
DELIVERY	delivery_id (PK), allocation_id (FK), pickup_time, delivery_time, delivery_status, notes	No
DONATION_HISTORY	donation_id (PK), allocation_id (FK), meals_served, completion_date, beneficiary_count, remarks	No

Non-simple / notable attributes

The schema primarily uses simple atomic attributes. Timestamp fields (created_at, request_date, allocation_date, pickup_time, delivery_time, prepared_time, available_until, completion_date) represent temporal values. Status and role fields use controlled enumerated values. The ER model does not identify a weak entity; all eight entities have their own primary key.

ER Diagram Snapshot

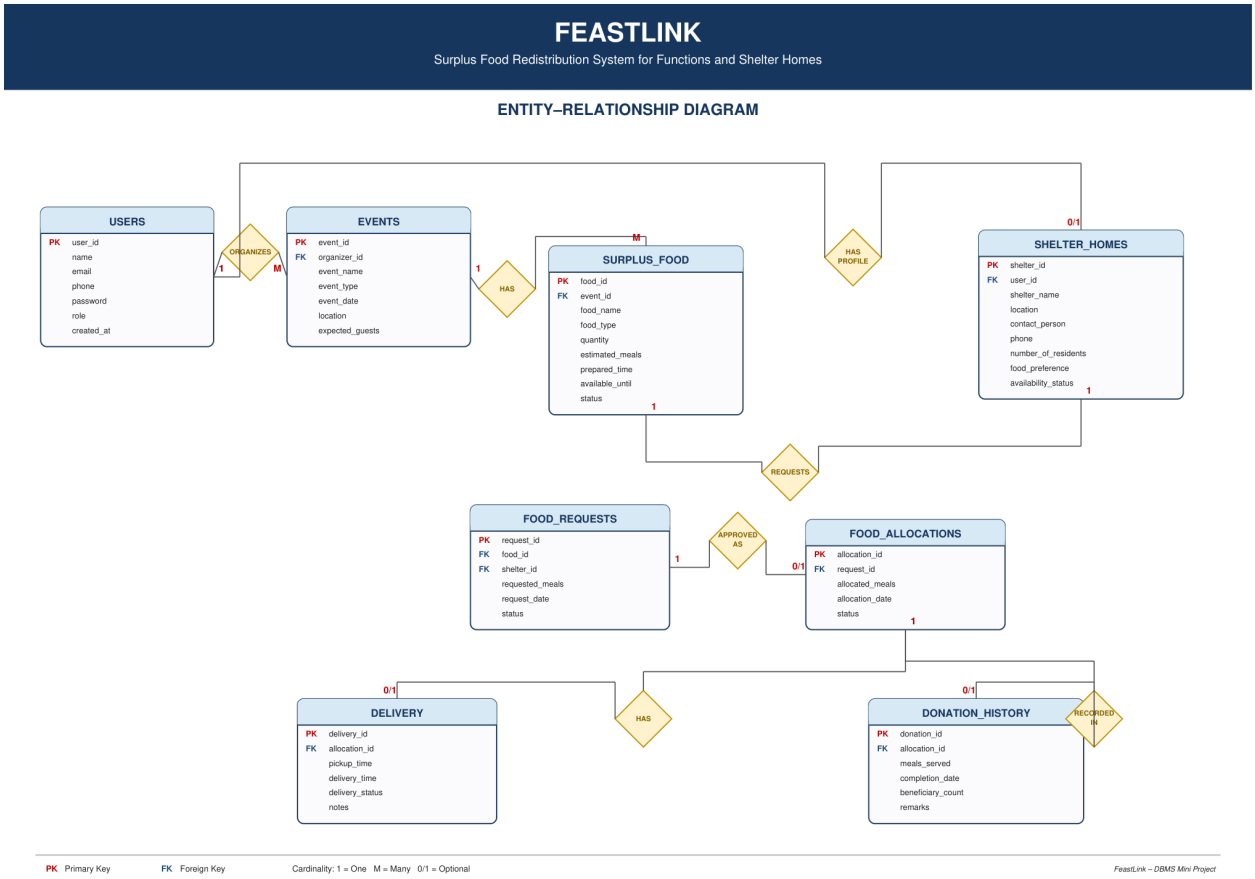


Figure: Original FEASTLINK Entity–Relationship Diagram

The ER diagram below is taken from the supplied FEASTLINK DBMS mini-project documentation (ER model page).

Relational Model

List of Relations

The following eight relations correspond to the eight principal ER entities. No additional associative relation outside the ER entities is introduced; FOOD_REQUESTS represents the request relationship with its own attributes and key.

USERS(user_id PK, name, email UNIQUE, phone, password, role, created_at)

EVENTS(event_id PK, organizer_id FK → USERS.user_id, event_name, event_type, event_date, location, expected_guests)

SHELTER_HOMES(shelter_id PK, user_id FK UNIQUE → USERS.user_id, shelter_name, location, contact_person, phone, number_of_residents, food_preference, availability_status)

SURPLUS_FOOD(food_id PK, event_id FK → EVENTS.event_id, food_name, food_type, quantity, estimated_meals, prepared_time, available_until, status)

FOOD_REQUESTS(request_id PK, food_id FK → SURPLUS_FOOD.food_id, shelter_id FK → SHELTER_HOMES.shelter_id, requested_meals, request_date, status)

FOOD_ALLOCATIONS(allocation_id PK, request_id FK UNIQUE → FOOD_REQUESTS.request_id, allocated_meals, allocation_date, status)

DELIVERY(delivery_id PK, allocation_id FK UNIQUE → FOOD_ALLOCATIONS.allocation_id, pickup_time, delivery_time, delivery_status, notes)

DONATION_HISTORY(donation_id PK, allocation_id FK UNIQUE → FOOD_ALLOCATIONS.allocation_id, meals_served, completion_date, beneficiary_count, remarks)

Relationship Summary

Parent	Cardinality	Child
USERS	1:M	EVENTS
USERS	1:0/1	SHELTER_HOMES
EVENTS	1:M	SURPLUS_FOOD
SURPLUS_FOOD	1:M	FOOD_REQUESTS
SHELTER_HOMES	1:M	FOOD_REQUESTS
FOOD_REQUESTS	1:0/1	FOOD_ALLOCATIONS
FOOD_ALLOCATIONS	1:0/1	DELIVERY
FOOD_ALLOCATIONS	1:0/1	DONATION_HISTORY

Schema Diagram

The schema is represented by the ER diagram on the preceding page and the relational definitions above. Foreign-key constraints preserve parent-child integrity; UNIQUE constraints on shelter user_id and the allocation-related request_id / allocation_id columns enforce the intended one-to-one or optional one-to-one relationships.

SQL DDL Statements

Concise DDL excerpts for the eight core relations in the feastlink database:

CREATE DATABASE IF NOT EXISTS feastlink; USE feastlink;

```
CREATE TABLE users (user_id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100) NOT NULL, email VARCHAR(150) UNIQUE NOT NULL, phone VARCHAR(15), password VARCHAR(255) NOT NULL, role ENUM('ORGANIZER','SHELTER_ADMIN','ADMIN') NOT NULL, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);

CREATE TABLE events (event_id INT AUTO_INCREMENT PRIMARY KEY, organizer_id INT NOT NULL, event_name VARCHAR(150) NOT NULL, event_type VARCHAR(100), event_date DATE NOT NULL, location VARCHAR(255) NOT NULL, expected_guests INT, FOREIGN KEY (organizer_id) REFERENCES users(user_id) ON DELETE CASCADE);

CREATE TABLE shelter_homes (shelter_id INT AUTO_INCREMENT PRIMARY KEY, user_id INT UNIQUE NOT NULL, shelter_name VARCHAR(150) NOT NULL, location VARCHAR(255), contact_person VARCHAR(100), phone VARCHAR(15), number_of_residents INT, food_preference VARCHAR(100), availability_status ENUM('AVAILABLE','UNAVAILABLE') DEFAULT 'AVAILABLE', FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE);

CREATE TABLE surplus_food (food_id INT AUTO_INCREMENT PRIMARY KEY, event_id INT NOT NULL, food_name VARCHAR(150) NOT NULL, food_type ENUM('VEG','NON_VEG','OTHER') DEFAULT 'VEG', quantity VARCHAR(100), estimated_meals INT NOT NULL, prepared_time DATETIME, available_until DATETIME, status
ENUM('AVAILABLE','REQUESTED','ALLOCATED','EXPIRED','COMPLETED') DEFAULT 'AVAILABLE', FOREIGN KEY (event_id) REFERENCES events(event_id) ON DELETE CASCADE);

CREATE TABLE food_requests (request_id INT AUTO_INCREMENT PRIMARY KEY, food_id INT NOT NULL, shelter_id INT NOT NULL, requested_meals INT NOT NULL, request_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP, status
ENUM('PENDING','APPROVED','REJECTED','CANCELLED') DEFAULT 'PENDING', FOREIGN KEY (food_id) REFERENCES surplus_food(food_id) ON DELETE CASCADE, FOREIGN KEY (shelter_id) REFERENCES shelter_homes(shelter_id) ON DELETE CASCADE);

CREATE TABLE food_allocations (allocation_id INT AUTO_INCREMENT PRIMARY KEY, request_id INT UNIQUE NOT NULL, allocated_meals INT NOT NULL, allocation_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP, status
ENUM('PENDING','CONFIRMED','COMPLETED','CANCELLED') DEFAULT 'PENDING', FOREIGN KEY (request_id) REFERENCES food_requests(request_id) ON DELETE CASCADE);

CREATE TABLE delivery (delivery_id INT AUTO_INCREMENT PRIMARY KEY, allocation_id INT UNIQUE NOT NULL, pickup_time DATETIME, delivery_time DATETIME, delivery_status ENUM('PENDING','PICKED_UP','IN_TRANSIT','DELIVERED','FAILED') DEFAULT 'PENDING', notes VARCHAR(500), FOREIGN KEY (allocation_id) REFERENCES food_allocations(allocation_id) ON DELETE CASCADE);

CREATE TABLE donation_history (donation_id INT AUTO_INCREMENT PRIMARY KEY, allocation_id INT UNIQUE NOT NULL, meals_served INT NOT NULL, completion_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP, beneficiary_count INT, remarks VARCHAR(500), FOREIGN KEY (allocation_id) REFERENCES food_allocations(allocation_id) ON DELETE CASCADE);
```

SQL DML Statements

Representative join, nested, aggregate, and write operations:

-- Join requests with food and shelter

```
SELECT fr.request_id, sf.food_name, sh.shelter_name, fr.requested_meals, fr.status
```

FROM food_requests fr JOIN surplus_food sf ON fr.food_id=sf.food_id

JOIN shelter_homes sh ON fr.shelter_id=sh.shelter_id;

-- Nested query: available listings above average meal estimate

```
SELECT food_id, food_name, estimated_meals FROM surplus_food WHERE status='AVAILABLE'
AND estimated_meals>(SELECT AVG(estimated_meals) FROM surplus_food);
```

-- Aggregate donation impact by shelter

```
SELECT sh.shelter_name, SUM(dh.meals_served), SUM(dh.beneficiary_count)
```

FROM donation_history dh JOIN food_allocations fa ON dh.allocation_id=fa.allocation_id

JOIN food_requests fr ON fa.request_id=fr.request_id JOIN shelter_homes sh ON fr.shelter_id=sh.shelter_id

GROUP BY sh.shelter_id, sh.shelter_name;

```
UPDATE food_requests SET status='APPROVED' WHERE request_id=2;
INSERT INTO donation_history (allocation_id,meals_served,beneficiary_count,remarks)
```

VALUES (2,100,100,'Donation completed');

NoSQL / Vector Database Commands

MongoDB collections: activity_logs and notifications in feastlink_logs. No vector database is documented as implemented.

```
use feastlink_logs;
db.createCollection("activity_logs"); db.createCollection("notifications");
db.activity_logs.insertOne({action:"USER_LOGIN",userId:1,details:{role:"ORGANIZER"},createdAt:new Date()});
db.notifications.insertOne({userId:2,title:"New food request",type:"request",read:false,createdAt:new Date()});
db.activity_logs.find({userId:1}).sort({createdAt:-1});
db.notifications.find({userId:2,read:false});
db.notifications.updateOne({userId:2,title:"New food request"},{$set:{read:true}});
db.notifications.deleteOne({title:"Test notification"});
```

Results

Resulting Tables and Testing Summary

The project handoff records successful MySQL and MongoDB connectivity checks and a demonstrated request-to-donation workflow. The following are observed test values documented in the handoff; they describe the test session and may change with database contents.

Test / outcome	Observed result
MySQL connectivity	MySQL 8.0 application-user login tested; feastlink database accessed using feastlink_app.
MongoDB connectivity	MongoDB 8.0.34 and Compass connected; feastlink_logs and its collections inspected.
Activity logging	USER_LOGIN and USER_LOGOUT activity documents observed.
Notification	A “New food request” notification observed with type=request and read=false.
Approval and allocation	Vegetable Pulao request for 100 meals approved; allocation_id 2, request_id 2, allocated_meals 100, status CONFIRMED.
Delivery lifecycle	Delivery progressed through PICKED_UP, IN_TRANSIT, and DELIVERED.
Donation history	100 meals served and 100 beneficiaries recorded for the test allocation.
Dashboard snapshot	2 food listings, 2 food requests, 150 meals served, 150 people reached, and 0 requests awaiting action at test time.

Next.js – Project Tree Structure

feastlink/

├── app/ # pages and API routes

│ ├── api/ # auth, food, requests, deliveries,

│ │ # history, dashboard

│ ├── dashboard/

│ └── history/

```
├── lib/
    ■   ■■■ db.ts                          # MySQL pool, MongoDB helper,
```

│ # audit() and notify()

├── public/

├── package.json

├── package-lock.json

└── .env.local # local secrets; do not commit

Sample Screenshots of Application Code and Demo

The supplied materials contain the ER diagram, relational schema, DDL, and a technical handoff, but no standalone application UI/code screenshots were provided in the accessible source set. To complete this evidence section in the final submission, insert authentic screenshots from the running application showing: (1) organizer dashboard/event listing, (2) surplus-food listing, (3) shelter browse/request page, (4) approval/allocation view, (5) delivery status, and (6) donation history/dashboard. Screenshots should be captured from the actual running project and must not be fabricated.

Implementation Notes

The documented DBMS implementation includes a view named v_food_distribution; trigger trg_request_approved; stored procedure approve_food_request; function meals_remaining; indexes on selected request, food, allocation, delivery, and event columns; and transaction handling in the donation-history API. The trigger updates food status when a request is approved and includes restoration logic for rejected/cancelled requests, subject to other requests for that food. Review the actual trigger definition before claiming its detailed branch behavior.

Evaluation Rubrics – Completion Status

#	Criteria	Distribution / scope	Status
1	Database Design	ER diagram, relational schema, keys, relationships, constraints	Implemented / documented
2	Database Creation	MySQL DDL, NoSQL database creation, technology justification	Implemented / documented
3	SQL Operations and Queries	DML, nested queries, joins; recursive/correlated query evidence	Partly documented; verify full query set
4	Advanced Database Concepts	Procedure/function, trigger, CTE; NoSQL feature RUD	Procedure, function, trigger, view and indexes documented; CTE/NoSQL CRUD evidence partly shown
5	Web Application and Database Integration	Next.js interface, backend/API and database integration	Implemented / documented in handoff
6	Authentication and Authorization	Frontend/backend authentication and minimum two roles with RBAC	Roles and login workflow documented; verify full RBAC enforcement
7	Documentation, Git/GitHub, Demonstration, Testing, Originality	Documentation, repository, demo, tests, individual contribution	Documentation and test workflow provided; GitHub/demo evidence not present in supplied materials

Team contribution summary (as documented): Saransh Mittal – organizer module, event creation, surplus-food management, MySQL schema and advanced SQL, organizer dashboard and integration. T Surya – shelter registration/profile and dashboard, food browsing and requests, MongoDB notifications/activity logs, testing and documentation.

Conclusion

FEASTLINK demonstrates a database-backed workflow for redistributing surplus food from functions and events to shelter homes. The MySQL schema models the core entities and relationships with keys, constraints, and cascade behavior, while MongoDB provides flexible support for activity logs and notifications. The documented application workflow includes event and food creation, shelter requests, approval, allocation, delivery tracking, and impact recording. Test documentation records a completed 100-meal donation flow and corresponding dashboard metrics. Further submission evidence should include authentic application screenshots, repository/commit proof, and verification of the complete advanced-query and role-based access-control requirements.

References / Project Materials

FEASTLINK_Complete_DBMS_Mini_Project.pdf – ER diagram, relational schema, and MySQL DDL.

FEASTLINK_Project_Handoff_and_Study_Guide.pdf – architecture, DBMS concepts, testing outcomes, workflow, and team responsibilities.

FEASTLINK_NextJS_App source archive – application implementation files.

CREATE DATABASE IF NOT EXISTS feastlink; USE feastlink;

```
CREATE TABLE users (user_id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100) NOT NULL, email VARCHAR(150) UNIQUE NOT NULL, phone VARCHAR(15), password VARCHAR(255) NOT NULL, role ENUM('ORGANIZER','SHELTER_ADMIN','ADMIN') NOT NULL, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);

CREATE TABLE events (event_id INT AUTO_INCREMENT PRIMARY KEY, organizer_id INT NOT NULL, event_name VARCHAR(150) NOT NULL, event_type VARCHAR(100), event_date DATE NOT NULL, location VARCHAR(255) NOT NULL, expected_guests INT, FOREIGN KEY (organizer_id) REFERENCES users(user_id) ON DELETE CASCADE);

CREATE TABLE shelter_homes (shelter_id INT AUTO_INCREMENT PRIMARY KEY, user_id INT UNIQUE NOT NULL, shelter_name VARCHAR(150) NOT NULL, location VARCHAR(255), contact_person VARCHAR(100), phone VARCHAR(15), number_of_residents INT, food_preference VARCHAR(100), availability_status ENUM('AVAILABLE','UNAVAILABLE') DEFAULT 'AVAILABLE', FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE);

CREATE TABLE surplus_food (food_id INT AUTO_INCREMENT PRIMARY KEY, event_id INT NOT NULL, food_name VARCHAR(150) NOT NULL, food_type ENUM('VEG','NON_VEG','OTHER') DEFAULT 'VEG', quantity VARCHAR(100), estimated_meals INT NOT NULL, prepared_time DATETIME, available_until DATETIME, status ENUM('AVAILABLE','REQUESTED','ALLOCATED','EXPIRED','COMPLETED') DEFAULT 'AVAILABLE', FOREIGN KEY (event_id) REFERENCES events(event_id) ON DELETE CASCADE);

CREATE TABLE food_requests (request_id INT AUTO_INCREMENT PRIMARY KEY, food_id INT NOT NULL, shelter_id INT NOT NULL, requested_meals INT NOT NULL, request_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP, status ENUM('PENDING','APPROVED','REJECTED','CANCELLED') DEFAULT 'PENDING', FOREIGN KEY (food_id) REFERENCES surplus_food(food_id) ON DELETE CASCADE, FOREIGN KEY (shelter_id) REFERENCES shelter_homes(shelter_id) ON DELETE CASCADE);

CREATE TABLE food_allocations (allocation_id INT AUTO_INCREMENT PRIMARY KEY, request_id INT UNIQUE NOT NULL, allocated_meals INT NOT NULL, allocation_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP, status ENUM('PENDING','CONFIRMED','COMPLETED','CANCELLED') DEFAULT 'PENDING', FOREIGN KEY (request_id) REFERENCES food_requests(request_id) ON DELETE CASCADE);

CREATE TABLE delivery (delivery_id INT AUTO_INCREMENT PRIMARY KEY, allocation_id INT UNIQUE NOT NULL, pickup_time DATETIME, delivery_time DATETIME, delivery_status ENUM('PENDING','PICKED_UP','IN_TRANSIT','DELIVERED','FAILED') DEFAULT 'PENDING', notes VARCHAR(500), FOREIGN KEY (allocation_id) REFERENCES food_allocations(allocation_id) ON DELETE CASCADE);

CREATE TABLE donation_history (donation_id INT AUTO_INCREMENT PRIMARY KEY, allocation_id INT UNIQUE NOT NULL, meals_served INT NOT NULL, completion_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP, beneficiary_count INT, remarks VARCHAR(500), FOREIGN KEY (allocation_id) REFERENCES food_allocations(allocation_id) ON DELETE CASCADE);
```